



Mini Project 5:

Deploy AI System on Azure (Mentoring and Completion)

The Scenario

A real-world problem inside the Mitsubishi group



BSI IT Service Desk

Day-in-the-life

BSI supports thousands of employees across Mitsubishi Indonesia subsidiaries. Every day the helpdesk receives hundreds of tickets — password resets, VPN issues, ERP errors, hardware requests.

Most are repetitive. Agents spend more time searching SOPs than solving real problems.



Repetitive workload

60–70% of tickets follow well-known resolution patterns from past cases.



Knowledge is scattered

Answers live in Confluence, PDFs, email threads — nothing is indexed.



Quality varies by agent

New hires take weeks before their response quality matches seniors.

Your Mission

Build your AI Assistant

Build a working **AI assistant** that helps BSI helpdesk agents resolve IT tickets faster — by **searching internal knowledge**, **classifying the ticket**, and **drafting a response** an agent can send.

Working

End-to-end demo on real-looking ticket inputs

Measured

Evaluated in AI Foundry with quality metrics

Affordable

Per-ticket cost calculated and justified

What You Will Build

Four working components by end of day



Knowledge Index

01

Azure AI Search index built over mocked SOP docs and resolved tickets, with hybrid + semantic ranker enabled.



RAG Responder

02

Retrieval-Augmented Generation flow using Azure OpenAI to draft an agent-ready reply with citations.



Ticket Classifier

03

Azure ML pipeline that routes the ticket into a category (network, access, hardware, ERP, other).



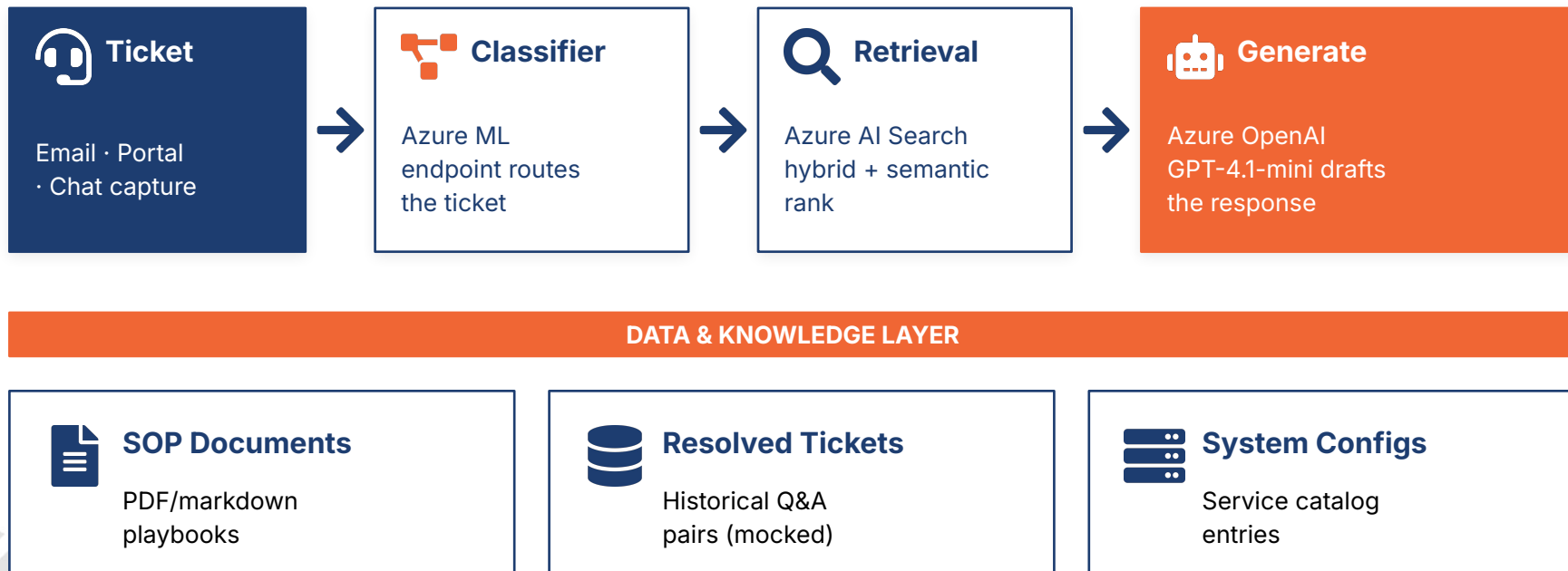
Eval & Monitor

04

AI Foundry evaluations for groundedness and relevance plus a basic cost & latency dashboard.

Solution Architecture

How the pieces fit together



Azure Services You'll Use

Your tech stack for this Mini Project



Resource Group

Logical container for all resources

Ch. 1



Azure OpenAI

GPT-4.1-mini for generation,
text-embedding-3-small for vectors

Ch. 2



Azure AI Search

Hybrid retrieval with semantic ranker

Ch. 2



Azure ML

Train and serve the ticket classifier

Ch. 3



AI Foundry

Prompt evaluation and quality
monitoring

Ch. 3



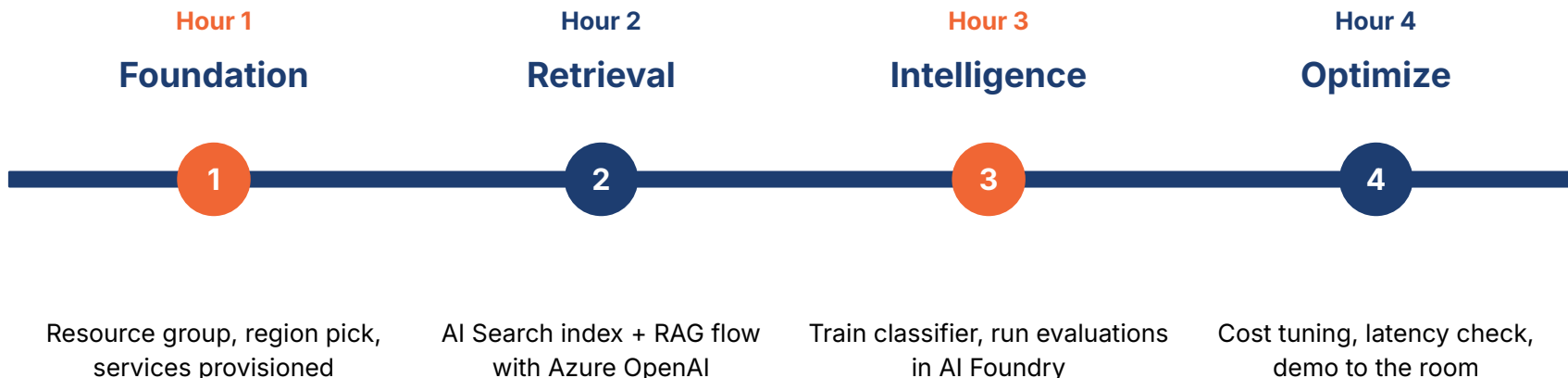
Blob Storage

Hold raw SOP and ticket files

Ch. 1

Your 4-hour Journey

Four phases · One end-to-end build



Plan to spend ~45 min building and ~15 min reflecting per phase. Don't skip the reflection — that's where cost discipline is learned.

Hour 1

Foundation & Setup

1 Create a resource group

Name it rg-bsi-itdesk-<your-initials>. Tag it for cost tracking.

2 Choose your region

Indonesia Central by default for regulatory. Fall back to Southeast Asia only when a service is not yet offered there.

3 Provision Azure OpenAI

Deploy gpt-4.1-mini (or gpt-4o-mini as alternative) and text-embedding-3-small. Note the quota.

4 Provision AI Search

Pick Basic tier. 1 replica, 1 partition is enough for the workshop.

5 Provision Azure ML workspace

Default compute. Add cluster nodes only when training requires it.

6 Upload starter data

Drop the mock SOP + tickets dataset into Blob Storage.

Hour 2

Build the retrieval pipeline

BUILD STEPS

- ✓ Chunk the SOP documents (~500 tokens, 50 token overlap)
- ✓ Generate embeddings with text-embedding-3-small
- ✓ Push to Azure AI Search with vector + keyword fields
- ✓ Enable semantic ranker on the index
- ✓ Wire up RAG: retrieve top-K → prompt GPT-4.1-mini → cite sources



Decisions you'll defend

What chunk size best fits your SOPs — short FAQs or long playbooks?

How many top-K to retrieve before you waste tokens?

Is semantic ranker worth the extra cost vs hybrid alone?

When should you say 'I don't know' instead of guessing?



Milestone: ask 'how do I reset my MFA token?' and get a grounded answer with citations

Hour 3

Add intelligence & measure quality



AZURE ML

Ticket classifier

Train a small classifier on the labelled tickets (network · access · hardware · ERP · other).

Wrap it in an Azure ML pipeline and deploy as a managed online endpoint.

Route the RAG prompt template based on category.



AI FOUNDRY

Evaluate quality

Groundedness

Does the answer stick to the retrieved SOPs?

Relevance

Does the answer actually address the ticket?

Coherence

Reads like a clear, professional reply?

Target: ≥80% groundedness · ≥70% relevance on the eval set

Hour 4

Optimize, test, and demo



Optimize

- ✓ Switch to smaller model where quality allows
- ✓ Reduce top-K and retry your eval
- ✓ Trim system prompt length



Test

- ✓ Run 20 mock tickets end-to-end
- ✓ Capture failure cases
- ✓ Measure latency per request



Present

- ✓ 5-minute individual demo
- ✓ Show cost-per-ticket math
- ✓ Explain region & model choices

End-state: working demo, eval scores, cost report, and a 1-slide architecture diagram — all packaged for individual submission.

Cost Decisions

Where your project's money will actually go

Cost lever	Cheap choice	Premium choice	Trade-off
Generation model	gpt-4.1-mini	gpt-4.1	Step-up cost; verify eval delta is worth it
Embedding model	text-embedding-3-small	text-embedding-3-large	Higher recall vs lower index cost
AI Search tier	Basic, 1 replica	Standard, 2+ replicas	Throughput, scale, SLA
Top-K retrieval	K = 3	K = 8	Context size · token cost
Semantic ranker	Off (hybrid only)	On (re-rank top 50)	Quality vs per-query add-on



Calculate per-ticket cost: (input + output) tokens × model price + (search ops × tier price). Target under USD 0.01 per ticket.

Model Trade-Offs

Generation models available in Indonesia Central

MODEL	RELATIVE COST	RELATIVE QUALITY
gpt-4.1 PREMIUM	100	100
gpt-4.1-mini RECOMMENDED	8	94
gpt-4.1-nano LIGHTEST	2	86
gpt-4o-mini ALTERNATIVE	6	90

RULE OF THUMB

Default to gpt-4.1-mini — better quality than gpt-4o at a fraction of the cost. Try **gpt-4.1-nano** for ultra-simple tickets, **gpt-4.1** only when the eval shows clear quality gains worth the cost.

The Constraints

Rules that force real engineering trade-offs

C1

 **Cost ceiling**

Cost per resolved ticket must stay under USD 0.01.

C2

 **Latency target**

End-to-end response in under 4 seconds.

C3

 **Honest answers**

Must say 'I don't know' when retrieval returns nothing relevant.

C4

 **Cite or it didn't happen**

Every response must show the SOP source it drew from.

What to Submit

Individual submission — four required artifacts

01

Source code

.ipynb or .zip

End-to-end notebook or code folder containing the RAG pipeline, classifier inference code, prompt templates, and a README with run instructions.

02

Evaluation report

.csv or .pdf · AI Foundry export

Groundedness, relevance, and coherence scores across at least 20 test queries. Include a screenshot of the AI Foundry evaluation run.

03

Cost & performance sheet

.xlsx or .md

Token math per request, per-ticket cost calculation, p50/p95 latency, plus a short justification for your region and model choice.

04

Architecture diagram

.png or .pdf (one page)

Visual showing all Azure services used, data flow, and the region each service is deployed in (Indonesia Central or Southeast Asia).



Package all four into one zip: **mini-project-5-{your-name}.zip** · Upload to the submission form.

Individual Scoring

How you will be evaluated

Working end-to-end pipeline

The pipeline runs on a fresh ticket and produces a cited response.



35 pts

Evaluation scores meet targets

Groundedness $\geq 80\%$, relevance $\geq 70\%$ on the AI Foundry eval set.



30 pts

Cost discipline & math

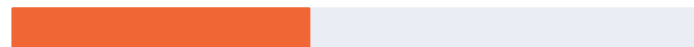
Per-ticket cost calculated, justified, and under USD 0.01.



20 pts

Region & model justification

Clear reasoning for Indonesia Central / SEA fallback and chosen model.



15 pts

Batas Pengumpulan: Hari ini, pukul 23.59 WIB
Pengumpulan melalui: Google Classroom



Thank You